

Calendly Clone

System Design Document · Scaler SDE Intern Fullstack Assignment

REFERENCE ARCHITECTURE · BUILD TO SPEC

Contents

1. High-Level Architecture

2. Tech Stack

3. Database Schema (ER + DDL)

4. REST API Specification

5. Slot Generation Algorithm

6. Concurrency & Double-Booking
7. Frontend Pages & Components

8. User Flow Diagrams

9. Folder Structure (Backend & Frontend)

10. Validation Rules & Edge Cases

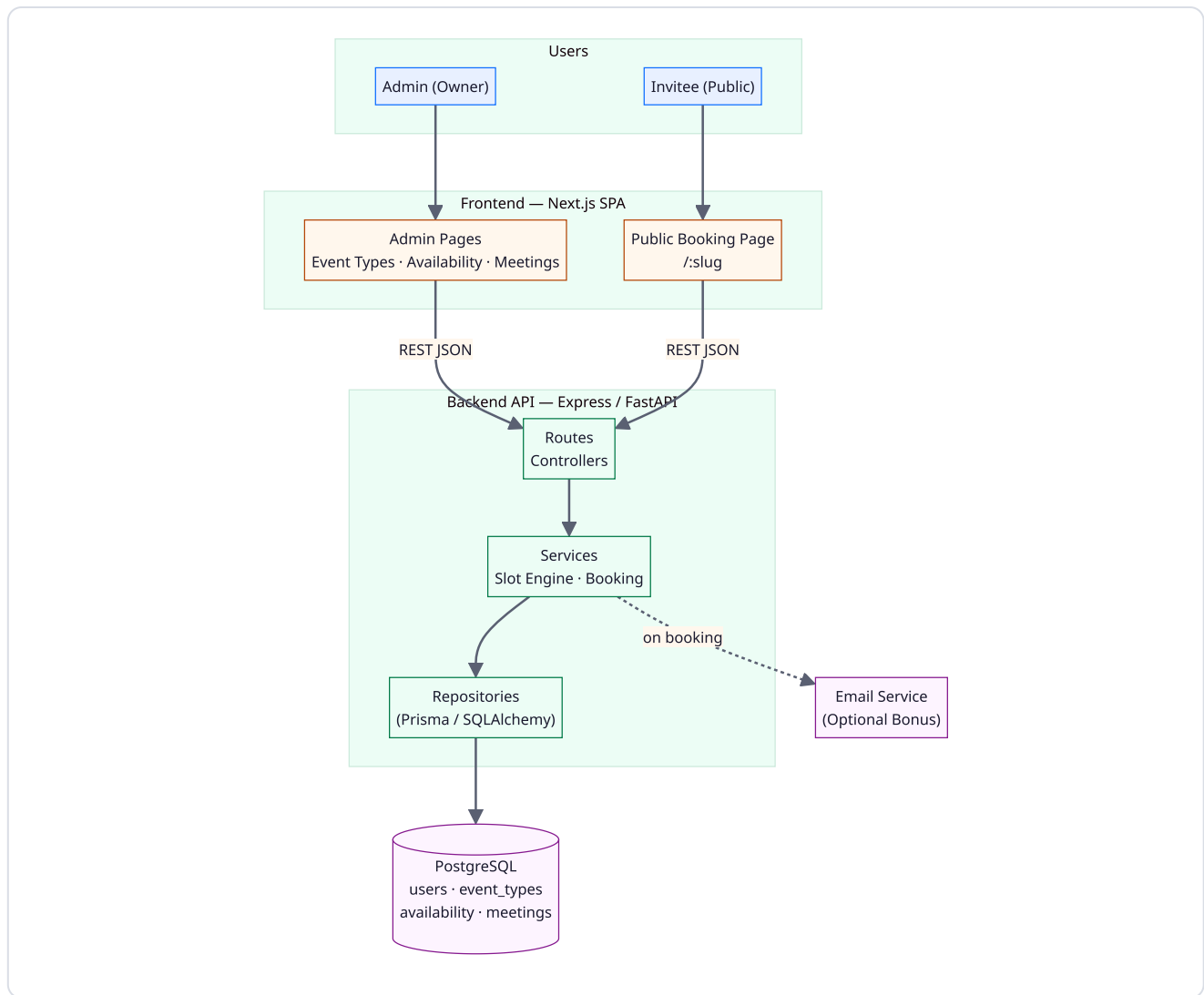
11. Seed Data & Deployment

12. README Checklist & Evaluation

13. 2-Day Build Plan

1. High-Level Architecture

A simple 3-tier monolith. Single SPA, single backend service, single relational database. No auth layer (per assignment: assume a default user is logged in).

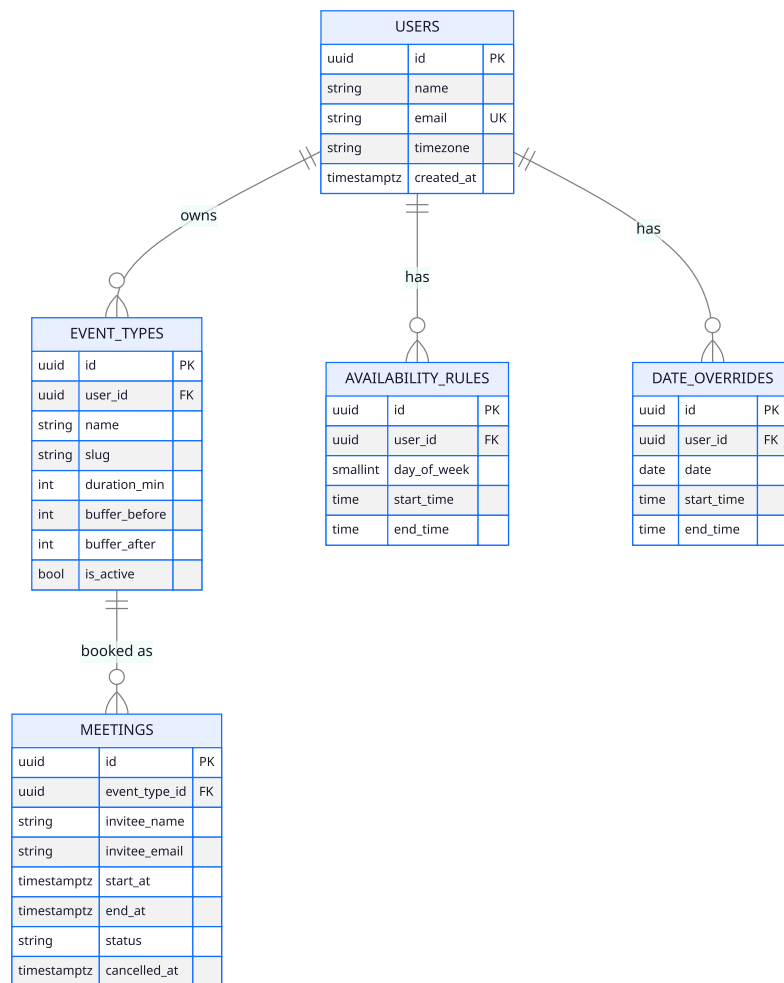


2. Tech Stack

Layer	Recommended Choice	Why
Frontend	Next.js 14 (App Router) + TypeScript + TailwindCSS	SPA + server components, Vercel-ready
UI Kit	shadcn/ui	Clean look matches Calendly aesthetic
Date / Time	luxon <i>or</i> date-fns-tz	Timezone & DST handled correctly
Backend	Node + Express + TypeScript <i>or</i> Python + FastAPI	Either is acceptable per brief
ORM	Prisma <i>or</i> SQLAlchemy	Schema-as-code, easy migrations
Database	PostgreSQL 15	Partial unique indexes (critical for booking)
Validation	Zod <i>or</i> Pydantic	Type-safe request validation
Deployment	Vercel (FE) · Render/Railway (BE) · Neon/Supabase (DB)	All free-tier friendly

3. Database Schema

3.1 Entity-Relationship Diagram



3.2 PostgreSQL DDL

```
-- 1. Users (single seeded user for this assignment)
CREATE TABLE users (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  name        VARCHAR(120) NOT NULL,
  email       VARCHAR(180) UNIQUE NOT NULL,
  timezone    VARCHAR(64)  NOT NULL DEFAULT 'Asia/Kolkata',
  created_at  TIMESTAMPTZ  NOT NULL DEFAULT NOW()
);

-- 2. Event Types
CREATE TABLE event_types (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id     UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  name        VARCHAR(120) NOT NULL,
  slug        VARCHAR(80)  NOT NULL,
  duration_min INT          NOT NULL CHECK (duration_min > 0),
  description  TEXT,
  buffer_before INT         NOT NULL DEFAULT 0,
  buffer_after INT         NOT NULL DEFAULT 0,
  is_active    BOOLEAN      NOT NULL DEFAULT TRUE,
  created_at  TIMESTAMPTZ  NOT NULL DEFAULT NOW(),
  UNIQUE (user_id, slug)
);

-- 3. Weekly availability rules (day_of_week: 0=Sun .. 6=Sat)
CREATE TABLE availability_rules (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id     UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  day_of_week SMALLINT NOT NULL CHECK (day_of_week BETWEEN 0 AND 6),
  start_time  TIME NOT NULL,
  end_time    TIME NOT NULL,
  CHECK (start_time < end_time)
);
CREATE INDEX idx_avail_user_day ON availability_rules(user_id, day_of_week);

-- 4. Date-specific overrides (BONUS)
CREATE TABLE date_overrides (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id     UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  date        DATE NOT NULL,
  start_time  TIME,    -- NULL = entire day unavailable
  end_time    TIME,
  UNIQUE (user_id, date)
);

-- 5. Meetings (bookings)
CREATE TABLE meetings (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  event_type_id UUID NOT NULL REFERENCES event_types(id) ON DELETE CASCADE,
  invitee_name VARCHAR(120) NOT NULL,
  invitee_email VARCHAR(180) NOT NULL,
  start_at     TIMESTAMPTZ  NOT NULL,    -- store UTC always
```

```
end_at      TIMESTAMPTZ  NOT NULL,
notes       TEXT,
status      VARCHAR(20)  NOT NULL DEFAULT 'confirmed',
cancelled_at TIMESTAMPTZ,
created_at   TIMESTAMPTZ NOT NULL DEFAULT NOW(),
CHECK (start_at < end_at)
);

-- CRITICAL: partial unique index prevents double-booking
CREATE UNIQUE INDEX uniq_active_booking
  ON meetings (event_type_id, start_at)
  WHERE status = 'confirmed';

CREATE INDEX idx_meetings_range
  ON meetings (event_type_id, start_at, end_at);
```

Critical: the partial unique index `uniq_active_booking` is your last line of defense against race conditions. Without it, two simultaneous bookings can both succeed.

4. REST API Specification

4.1 Admin — Event Types

Method	Path	Purpose
GET	/api/event-types	List all event types
POST	/api/event-types	Create new event type
GET	/api/event-types/:id	Get one event type
PATCH	/api/event-types/:id	Update event type
DELETE	/api/event-types/:id	Delete event type

4.2 Admin — Availability

Method	Path	Purpose
GET	/api/availability	Get weekly schedule + timezone
PUT	/api/availability	Replace entire weekly schedule

```
// PUT /api/availability body
{
  "timezone": "Asia/Kolkata",
  "rules": [
    { "day_of_week": 1, "start_time": "09:00", "end_time": "17:00" },
    { "day_of_week": 2, "start_time": "09:00", "end_time": "17:00" },
    { "day_of_week": 3, "start_time": "09:00", "end_time": "17:00" },
    { "day_of_week": 4, "start_time": "09:00", "end_time": "17:00" },
    { "day_of_week": 5, "start_time": "09:00", "end_time": "13:00" }
  ]
}
```

4.3 Public — Booking

Method	Path	Purpose
GET	/api/public/:slug	Public event-type info
GET	/api/public/:slug/month? year=2026&month=5&tz=...	Dates in month that have any slot
GET	/api/public/:slug/slots?date=YYYY-MM-DD&tz=...	Available slots on a date
POST	/api/public/:slug/book	Create a booking

```
// GET /api/public/30min/slots?date=2026-05-22&tz=Asia/Kolkata
{
  "date": "2026-05-22",
  "timezone": "Asia/Kolkata",
  "slots": ["09:00", "09:30", "10:00", "10:30", "11:30"]
}

// POST /api/public/30min/book
{
  "start_at": "2026-05-22T09:00:00+05:30",
  "invitee_name": "Asha Verma",
  "invitee_email": "asha@example.com",
  "notes": ""
}

// → 201 Created
{
  "id": "uuid", "start_at": "...", "end_at": "...",
  "event_type": { "name": "30 Minute Meeting", "duration_min": 30 }
}

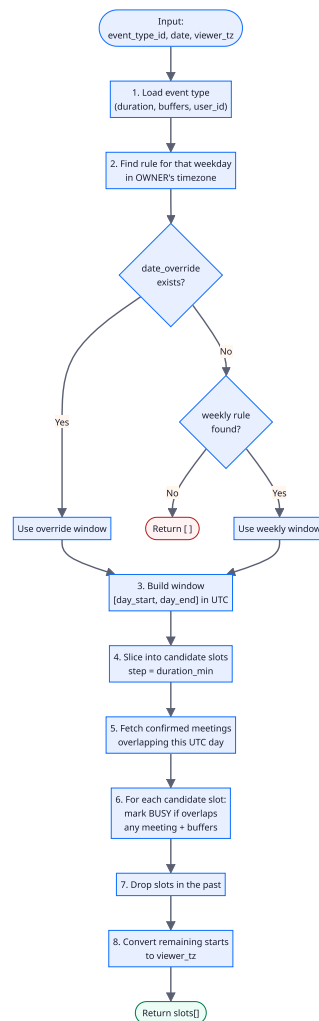
// → 409 Conflict (slot was just taken)
{ "error": "SLOT_TAKEN", "message": "This time was just booked. Pick another." }
```

4.4 Admin — Meetings

Method	Path	Purpose
GET	/api/meetings?filter=upcoming past	List meetings
POST	/api/meetings/:id/cancel	Cancel a meeting

5. Slot Generation Algorithm

The heart of the system. Given an event type + date + viewer timezone, return all bookable start times.

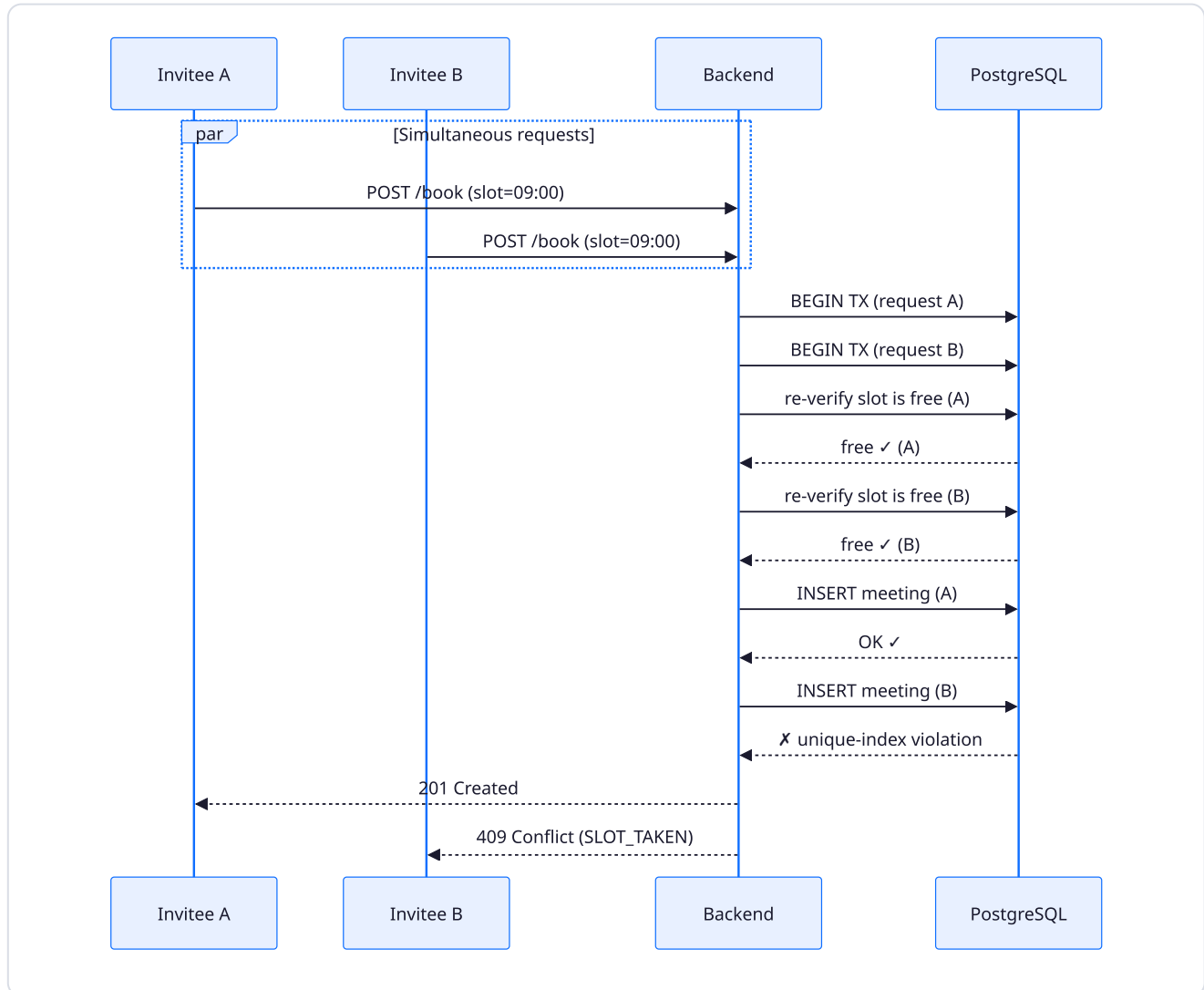


Overlap test (memorize this): `slot_start < meeting_end AND slot_end > meeting_start` — two intervals overlap iff each starts before the other ends.

Timezone rule: Store everything in UTC in the database. Convert to local time only at the edges (when accepting input from the user, and when rendering for display). Never do manual `+offset` arithmetic — let luxon / pytz handle DST.

6. Concurrency & Double Booking

Two invitees can hit `POST /book` at the same millisecond. The system must accept exactly one.



Two layers of defense:

1. **Application layer** — wrap the slot re-check + insert in a single DB transaction.
2. **Database layer** — the partial unique index `uniq_active_booking` guarantees only one confirmed row per `(event_type, start_at)`. The race loser receives a unique-violation error → API returns HTTP 409.

7. Frontend Pages & Components

Admin Routes

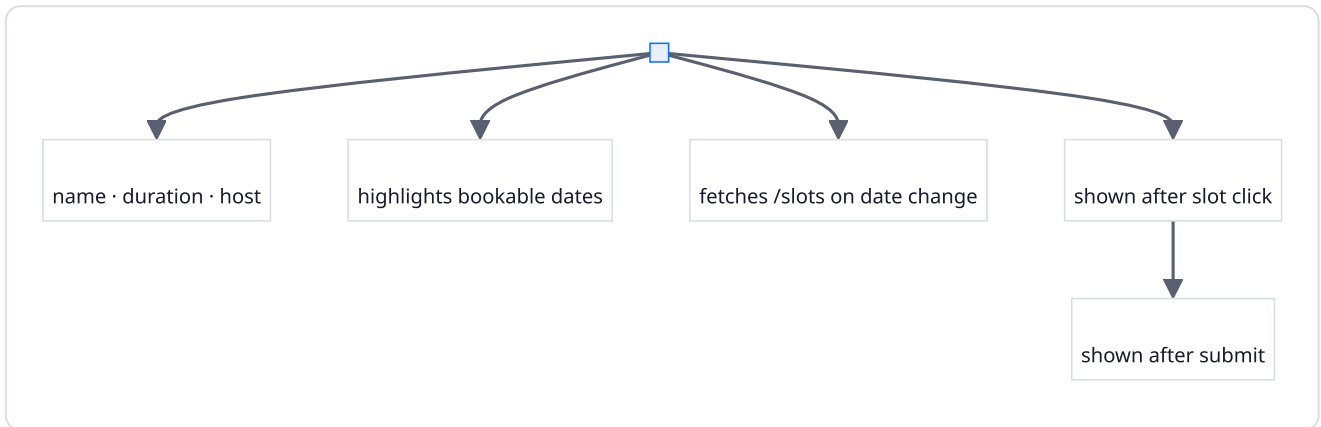
- `/` → redirect to `/event-types`
- `/event-types` — list + CRUD
- `/availability` — weekly grid + tz

Public Routes

- `/:slug` — calendar + slots + form
- `/:slug/confirmed/:meetingId`

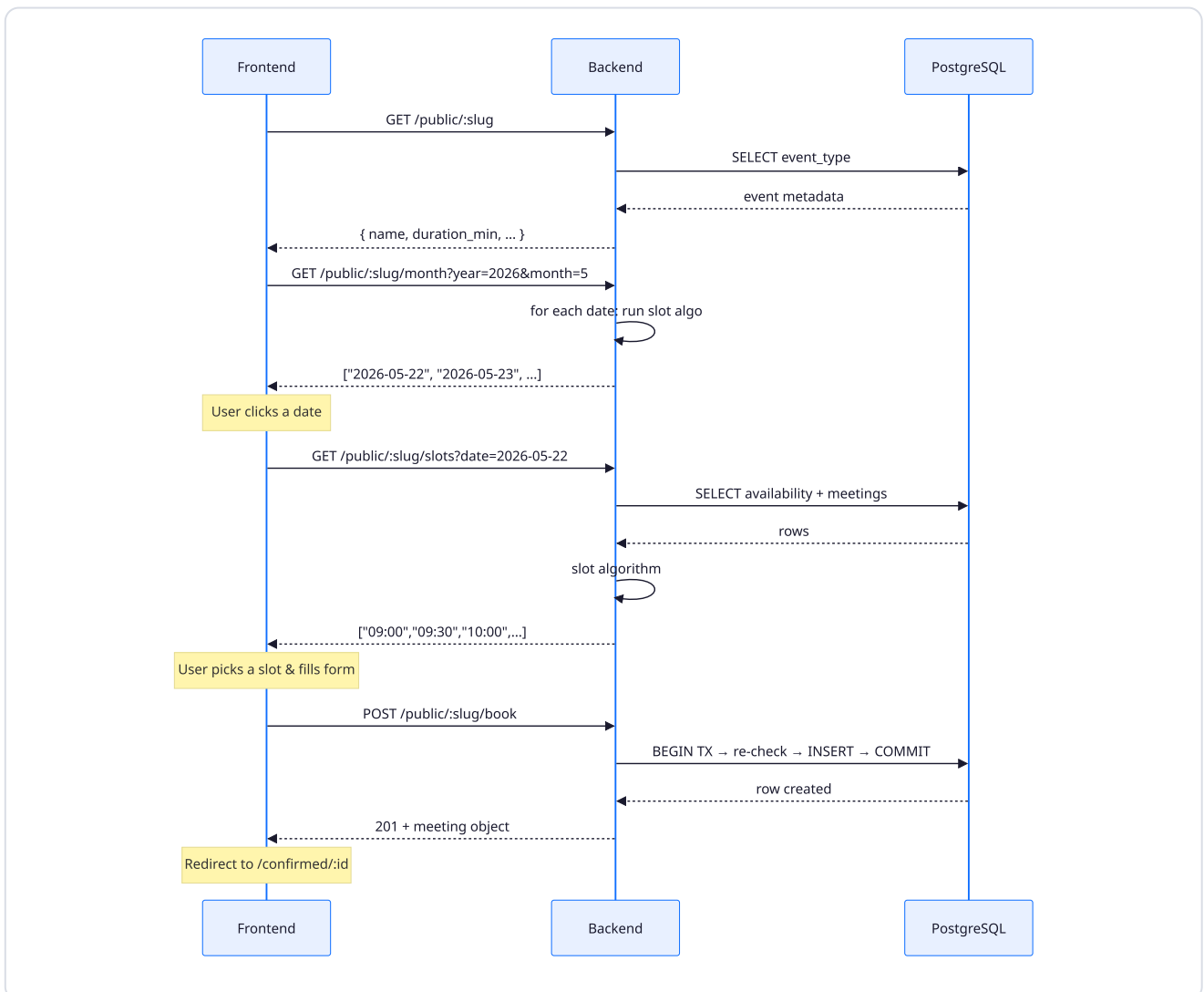
- `/meetings?tab=upcoming|past`

Booking Page Component Tree

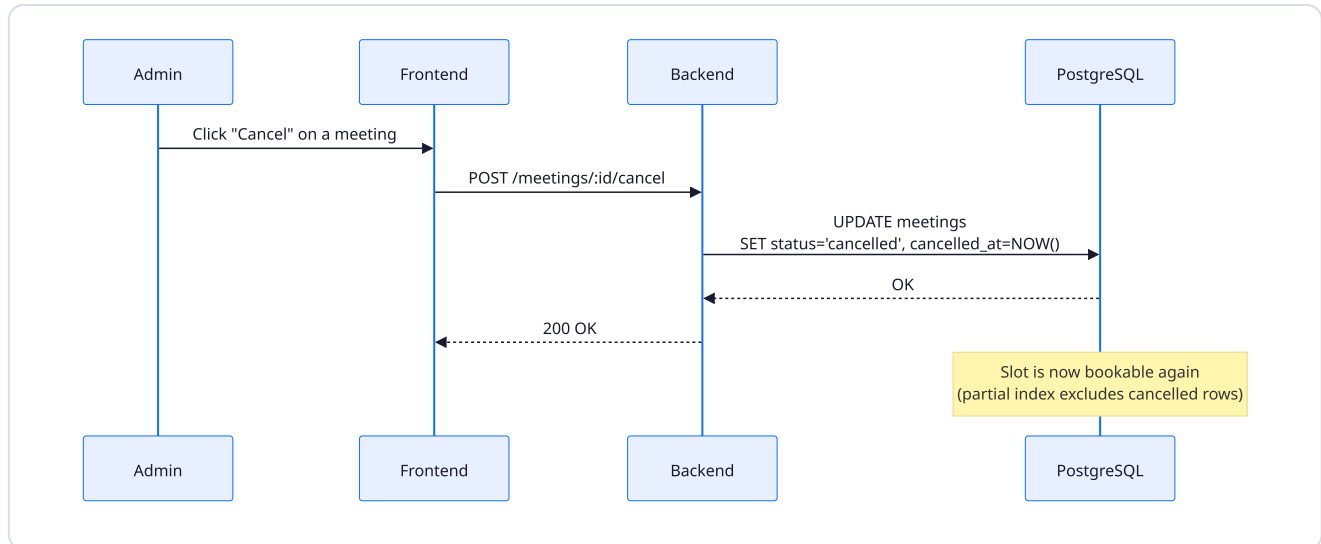


8. User Flow Diagrams

8.1 Booking Flow (Happy Path)



8.2 Cancellation Flow



9. Folder Structure

Backend (Node + Express)

```

backend/
├── src/
│   ├── index.ts
│   ├── routes/
│   │   ├── eventTypes.routes.ts
│   │   ├── availability.routes.ts
│   │   ├── meetings.routes.ts
│   │   └── public.routes.ts
│   ├── controllers/      ← thin
│   ├── services/         ← business logic
│   │   ├── slotEngine.ts ★
│   │   └── bookingService.ts
│   ├── repositories/     ← Prisma calls
│   ├── lib/
│   │   ├── prisma.ts
│   │   └── time.ts
│   ├── validators/       ← zod schemas
│   └── middlewares/
│       └── errorHandler.ts
├── prisma/
│   ├── schema.prisma
│   └── seed.ts
└── package.json
  
```

Frontend (Next.js App Router)

```

frontend/
├── app/
│   ├── (admin)/
│   │   ├── event-types/page.tsx
│   │   ├── availability/page.tsx
│   │   └── meetings/page.tsx
│   ├── [slug]/
│   │   ├── page.tsx
│   │   └── confirmed/[id]/page.tsx
│   └── layout.tsx
├── components/
│   ├── ui/                ← shadcn
│   ├── MonthCalendar.tsx
│   ├── TimeSlotList.tsx
│   ├── BookingForm.tsx
│   ├── EventTypeCard.tsx
│   └── AvailabilityGrid.tsx
├── lib/
│   ├── api.ts
│   └── time.ts
└── package.json
  
```

Separation of concerns: routes → controllers → services → repositories. Controllers never touch the DB. All business logic lives in `services/`.

10. Validation Rules & Edge Cases

Field / Scenario	Rule
<code>event_type.slug</code>	lowercase, kebab-case, 1–80 chars, unique per user
<code>duration_min</code>	positive integer (suggest 15/30/45/60/90/120)
availability	<code>start_time < end_time</code> ; no overlapping rules same day
<code>booking.start_at</code>	Re-run slot algo server-side ; never trust client
<code>booking.email</code>	RFC-valid email
timezone	valid IANA name (e.g. <code>Asia/Kolkata</code>)
Race condition	409 on unique-violation, friendly error to user
Booking in past	Reject server-side
DST transitions	Use luxon/pytz; never manual offset math
Cancelled meeting	Slot becomes free again (index allows it)
Delete event type	Cascade meetings <i>or</i> block if upcoming exist — document choice in README
End-of-day slot	Ensure <code>slot_end ≤ window_end</code>

11. Seed Data & Deployment

11.1 Seed Script (Prisma)

```
// prisma/seed.ts
import { PrismaClient } from "@prisma/client";
const prisma = new PrismaClient();

async function main() {
  const user = await prisma.user.create({
    data: {
      name: "Demo User",
      email: "demo@calendly-clone.com",
      timezone: "Asia/Kolkata",
      eventTypes: { create: [
        { name: "15 Minute Meeting", slug: "15min", duration_min: 15 },
        { name: "30 Minute Meeting", slug: "30min", duration_min: 30 },
        { name: "60 Minute Meeting", slug: "60min", duration_min: 60 },
      ]},
      availabilityRules: { create: [1,2,3,4,5].map(d => ({
        day_of_week: d, start_time: "09:00", end_time: "17:00",
      })))},
    },
  });
  // + insert 2-3 sample upcoming meetings
}
main().finally(() => prisma.$disconnect());
```

11.2 Deployment Matrix

Component	Service	Notes
Frontend	Vercel	<code>next build</code> , set <code>NEXT_PUBLIC_API_URL</code>
Backend	Render / Railway	Web service on <code>\$PORT</code>
Database	Neon / Supabase	Free Postgres, copy <code>DATABASE_URL</code>

11.3 Environment Variables

```
DATABASE_URL=postgres://user:pass@host:5432/db
PORT=4000
FRONTEND_ORIGIN=https://your-app.vercel.app # for CORS allowlist
DEFAULT_USER_EMAIL=demo@calendly-clone.com # seeded user
```

12. README Checklist & Evaluation Mapping

12.1 README must contain

- 1. Tech stack used
- 2. Local setup: `git clone` → `.env` → `npm install` → `prisma migrate dev` → `prisma db seed` → `npm run dev`
- 3. How to run frontend + backend together
- 4. Deployed URLs (frontend + backend)
- 5. **Assumptions made** (no auth, slot granularity, etc.)
- 6. **Database schema diagram** (image or ASCII)
- 7. **API documentation** (Postman collection or markdown table)
- 8. Screenshots: admin pages + booking flow + confirmation

12.2 How this design satisfies each evaluation criterion

Criterion	Where it's addressed
Functionality	All 4 core features in §4 (APIs) and §7 (pages)
UI / UX	§7 page structure; shadcn/ui for Calendly look
Database Design	§3 — normalized schema, FK constraints, indexes
Code Quality	§10 validation, error handler middleware, typed end-to-end
Code Modularity	§9 routes / controllers / services / repositories split
Code Understanding	Student must be able to explain §5 (slot algo) & §6 (concurrency)

13. Suggested 2-Day Build Plan

Day 1 (~8 hrs)

- Repos, DB, Prisma schema, seed (1h)
- Backend: event-types CRUD + availability (2h)
- Backend: slot algorithm + unit tests (2h)
- Backend: booking endpoint with TX + 409 (1h)
- Frontend admin pages (event types + availability) (2h)

Day 2 (~8 hrs)

- Frontend: public booking page (calendar + slots + form) (3h)
- Frontend: confirmation + meetings page (2h)
- Polish UI to match Calendly look (1.5h)
- Deploy + write README + screenshots (1.5h)

End of system design. Build to spec, then iterate on polish.
Generated as the reference architecture for the SDE Intern Fullstack Assignment.